An Empirical Study for Inversions-Sensitive Sorting Algorithms

Amr Elmasry[1,2,*] and Abdelrahman Hammad[1]

[1] Dept. of Computer Engineering and Systems, Alexandria University, Egypt
[2] Faculty of Science and Information Technology, Al-Zaytoonah University, Jordan

Abstract. We study the performance of the most practical internal adaptive sorting algorithms. Experimental results show that adaptive AVL sort performs the least number of comparisons unless the number of inversions is fewer than 1%. In such case, Splaysort performs the fewest number of comparisons. On the other hand, the running time of Quicksort is superior unless the number of inversions is fewer than 1.5%. In such case, Splaysort consumes the smallest running time.

1 Introduction

A sorting algorithm is considered adaptive if it performs better for sequences having some existing order. One of the main measures of presortedness is the number of inversions in the input sequence Inv, that is the number of pairs in the wrong order. For an adaptive sorting algorithm to be optimal with respect to the number of inversions, its running time should be in $O(n \log \frac{Inv}{n} + n)$ [6].

Among the sorting algorithms optimal with respect to the number of inversions, Splitsort [8] and adaptive Heapsort [9] are the most promising from the practical point of view. Splaysort, sorting by repeated insertion in a splay tree [11], was proved to be optimal with respect to the number of inversions [2]. Moffat et al. [10] performed experiments showing that Splaysort is practically efficient. See [5] as a nice survey of adaptive sorting algorithms. A later study, oriented towards improving the number of comparisons for inversions-optimal sorting algorithms, introduces adaptive AVL sort [4].

On the other hand, Quicksort introduced by Hoare [7] is considered the most practical sorting algorithm. Several empirical studies illustrated that Quicksort is very efficient in practice [13]. The Quicksort algorithm and its variants also demonstrated an efficient performance when applied to nearly sorted lists [3]. A very recent result of Brodal et al. [1] shows that the expected number of swaps performed by randomized Quicksort is $O(n \log \frac{Inv}{n} + n)$.

In this paper, we are interested in demonstrating a practical study of such inversions-optimal sorting algorithms. Our objective is to conclude which of these

* This work was done while the author was visiting Al-Zaytoonah University. Corresponding author. E-mail address: elmasry@alexeng.edu.eg

S.E. Nikoletseas (Ed.): WEA 2005, LNCS 3503, pp. 597–601, 2005.
© Springer-Verlag Berlin Heidelberg 2005

algorithms would be a good candidate in practice, and which one we would use under different circumstances. We also compare the performance of these algorithms with Quicksort.

2 Experimental Settings

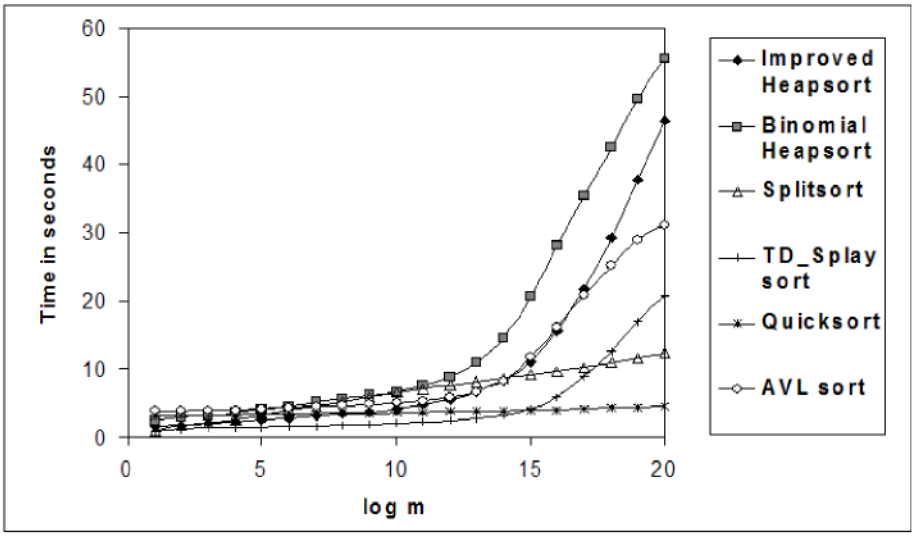
We implemented three versions of adaptive Heapsort; the basic implementation that uses a binary heap, an improved version for heap operations [9], and another variation that uses binomial queues. For Splitsort, we implemented two versions; the first uses an auxiliary array of pointers [8], while the second relies on linked structures. It turns out that the running time of the array-based Splitsort is less than the linked implementation. Two variations of the Splaysort algorithm were implemented, one using bottom-up splaying and the other using top-down(TD) splaying [10]. It turns out that the running time for the top-down version is much less. Finally we implemented adaptive AVL sort as in [4].

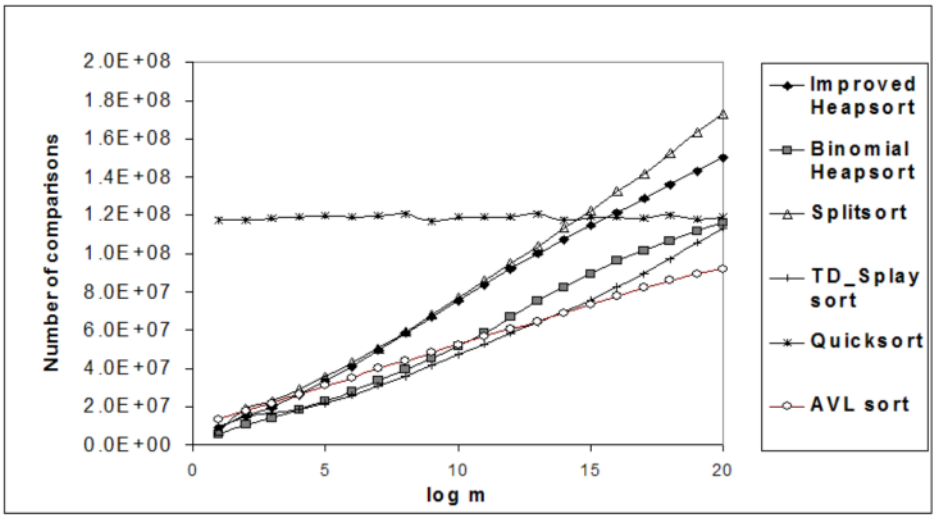
In all the experiments, we start with a sorted sequence $< 1, 2, \ldots, n >$ and perform two phases of permutations. For a given value of a parameter m, we want to permute the sorted sequence to have at most $n \cdot m$ with expected $\approx n \cdot m/2$ inversions. For the first phase, the sequence is broken into consecutive $\lceil n/m \rceil$ blocks, almost equally-sized. The elements of each block are randomly permuted, for a total of at most $n \cdot m/2$ inversions. For the second phase, the sorted sequence is broken into m consecutive blocks, almost equally-sized. From each block we select one element at random, and these m elements are then randomly permuted, for at most another $n \cdot m/2$ inversions. Hence, the resulting average number of inversions per element will be at most m.

Note that the way we produce the input guarantees that the elements of the input sequence are distinct. It is still an issue to check the performance of the given algorithms when there are duplicate elements. As an example, the adaptive properties of the used version of Heapsort are only correct when applied to distinct elements. Another issue is that we applied the algorithms for integer sorting; all our elements are 4-byte integers. The performance may be different if we apply the algorithms to sort real numbers or other data types.

We fixed n at 2^{22}. All the experiments were performed on an Athlon750 machine with 384 MB RAM and 256 KB cache, running Windows 2000 platform. With such memory capacity, there was no need to use virtual memory. All the algorithms are implemented in C++ using Borland C++ builder version 5.

Two outcomes are measured; the number of comparisons and the running time. The measured running times are CPU times, ignoring the time for I/O operations. For each point represented in the graphs, the average of 10 sample runs was taken. We first proceeded by investigating different alternatives and implementation issues for each algorithm. Then, we concluded by selecting the best method from each algorithm and comparing them with each other.





3 Experimental Results

One of our main observations for the running time of the algorithms that require linked structures is that the dynamic allocations of such structures is time consuming. We noticed that more than 15% of the time of such algorithms (Heapsort, Splaysort and AVL sort) was used for such memory allocations. Our solution was to allocate a chunk of memory at the beginning of the algorithm. It was not hard to decide the memory requirement for Heapsort and

Splaysort. For AVL sort, the theoretical results in [4] indicated that the number of combines (extra nodes needed throughout the algorithm) is linear. Working through these proofs, we came up with the exact constant which is at most $3n$ combines.

Comparing the running time of the other algorithms with Quicksort, these algorithms were better for low inversions. As the number of inversions increases, Quicksort is preferable for both the running time and the storage requirements. As illustrated by the graph, TD-Splaysort shows the best performance when the number of inversions is low. It is better than Quicksort as long as the number of inversions per element is less than 2^{15} ($n = 2^{22}$). Splitsort, although not the best with low inversions, is better than Splaysort for high inversions, while it is not much worse than Splaysort with low inversions. In addition, it has less storage requirements. It is also interesting to demonstrate that Quicksort is adaptive with respect to time, this is a consequence of the result of [1] that Quicksort is optimally adaptive with respect to the number of swaps.

Experimental results show that the number of cache misses, and in effect the running time, is related to the number of inversions and the storage complexity of the underlying algorithm. For the algorithms with large storage requirements, the curves have sharp bends when the inversions increase (at $m \approx 13$). This illustrates that these algorithms are adaptive with respect to cache misses.

Regarding the number of comparisons, the adaptive algorithms under study were noticeably much better than Quicksort for low number of inversions. Some of the algorithms perform less comparisons even for random lists. Empirical results show that all the adaptive algorithms under study perform $c \cdot n \log \frac{Inv}{n} + O(n)$ comparisons, with c between 1 and 2. For the AVL sort, $c \approx 1$.

References

1. G. Brodal, R. Fagerberg and G. Moruz. *On the adaptiveness of quicksort.* 7th (ALENEX) Workshop on Algorithm Engineering and Experiments (2005).
2. R. Cole. *On the dynamic finger conjecture for splay trees. Part II: The proof.* SIAM J. Computing **30** (2000), 44-85.
3. R. Cook and J. Kim. *Best sorting algorithms for nearly sorted lists.* Commun. ACM **23** (1980), 620-624.
4. A. Elmasry. *Adaptive sorting with AVL trees.* 3rd IFIP-WCC International Conference on Theoretical Computer Science (2004), 315-324.
5. V. Estivill-Castro and D. Wood. *A survey of adaptive sorting algorithms.* ACM Computing Surveys **24(4)** (1992), 441-476.
6. L. Guibas, E. McCreight, M. Plass and J. Roberts. *A new representation of linear lists.* 9th ACM (STOC) Symposium on Theory of Computing **9** (1977), 49-60.
7. C. Hoare. *Algorithm 64: Quicksort.* Commun. ACM **4(7)** (1961), 321.
8. C. Levcopoulos and O. Petersson. *Splitsort - An adaptive sorting algorithm.* Information Processing Letters **39** (1991), 205-211.
9. C. Levcopoulos and O. Petersson. *Adaptive Heapsort.* J. Alg. **14** (1993), 395-413.
10. A. Moffat, G. Eddy and O. Petersson. *Splaysort: fast, versatile, practical.* Softw. Pract. and Exper. **126(7)** (1996), 781-797.

11. D. Sleator and R. Tarjan. *Self-adjusting binary search trees.* J. ACM **32(3)** (1985), 652-686.
12. J. Vuillemin. *A unifying look at data structures.* Commu. ACM **23** (1980), 229-239.
13. R. Wainwrigh. *A class of sorting algorithms based on quicksort.* Commun. ACM **28(4)** (1985), 396-402.